

Introduction

Every so often a problem shows up that looks like it needs a fancy data structure — a heap, a sorted map, maybe even a segment tree — and then turns out to be a two-pointer exercise wearing a disguise. Length of Longest Subarray With at Most K Frequency (LeetCode 2958) is exactly that kind of problem, and it's a favorite in interviews for a specific reason: it tests whether you actually understand *why* sliding window works, not just whether you've memorized the template.

The core idea — find the longest stretch of a sequence where no value repeats too often — isn't just an interview trick. It's the same shape behind rate limiters, spam filters, and anomaly-detection windows in real production systems. Once you internalize this pattern, you'll start recognizing it everywhere: "longest/shortest run under a running constraint" is a signal to reach for two pointers and a count map, before anything more exotic.

Problem Overview

You're given an array of integers and an integer `k`. You need to find the length of the longest contiguous subarray such that no single value appears more than `k` times inside it.

In plain terms: slide a window across the array, and as long as every distinct number inside that window shows up `k` times or fewer, the window is "legal." Your job is to find the biggest legal window that ever existed as you swept across the array.

Example

Consider the array:

```
[1, 2, 3, 1, 2, 3, 1, 2], k = 2
```

The first six elements — `[1, 2, 3, 1, 2, 3]` — form a valid window. Each value (`1`, `2`, `3`) appears exactly twice, which is within the limit of `k = 2`.

Add the seventh element (`1`), and now `1` appears three times. The window is no longer legal, so it has to shrink from the left until it's valid again.

The answer for this array is `6` — the largest legal window we found.

Intuition

The naive instinct is: "just check every possible subarray." Pick a start, pick an end, count the frequency of every element in between, and see if it's legal. That works, but think about what happens as you slide the end pointer forward by one position: almost nothing about the window changed. You added exactly one value. Yet the brute-force approach throws away everything it knew and recounts from scratch.

That's the real waste — not the comparison itself, but discarding work you already did. An experienced engineer's first move here is to ask: "What do I already know that I'm about to recompute?" The answer is the frequency counts of everything currently in the window. If you keep those counts updated incrementally instead of recalculating them, you can process each element exactly once.

That leads directly to the sliding window pattern: two pointers, `left` and `right`, define the window's boundaries. A dictionary tracks how many times each value currently appears inside that window. The `right` pointer always moves forward, adding one new element at a time. The `left` pointer only moves forward when the window becomes illegal — that is, when some value's count exceeds `k`.

Here's the part that trips people up: shrinking is a `while` loop, not an `if`. If duplicates pile up quickly, dropping a single element from the left might not be enough to make the window legal again. You need to keep shrinking until the count is actually back at or below `k`.

Brute Force Solution

Idea: For every pair of start and end indices, count the frequency of every value in that subarray from scratch, and check if the maximum frequency is at most `k`.

Algorithm: 1. For every possible `start` index. 2. For every possible `end` index greater than or equal to `start`. 3. Count frequencies of all elements between `start` and `end`. 4. If the max frequency is at most `k`, update the best length.

Advantages: Trivial to reason about and implement; hard to get wrong.

Disadvantages: Recomputes counts that barely changed between adjacent windows. Quadratic number of subarrays, and counting each one costs additional time — this becomes far too slow once the input grows.

Time complexity: $O(n^3)$ in the naive form ($O(n^2)$ subarrays $\times O(n)$ to count each), or $O(n^2)$ if you count incrementally per `start`. Either way, it doesn't scale.

Space complexity: $O(n)$ for the frequency map used per subarray check.

Optimal Solution

The optimal solution uses a growing-and-shrinking window with a single frequency dictionary.

1. Start with an empty `counts` dictionary, `left = 0`, and `best = 0`.
2. For every `right` position, add the value at `right` into `counts`, incrementing its count.
3. While the count of that value exceeds `k`, remove the value at `left` from `counts` and move `left` forward — repeat until the window is legal again.
4. After each shrink step, the window is guaranteed legal. Compute its size as `right - left + 1`, and update `best` if it's larger.

Step	right	Value added	Window	Legal?	Best so far
1	0	1	[1]	✓	1
2	1	2	[1,2]	✓	2
3	2	3	[1,2,3]	✓	3
4	3	1	[1,2,3,1]	✓	4
5	4	2	[1,2,3,1,2]	✓	5
6	5	3	[1,2,3,1,2,3]	✓	6
7	6	1	count of 1 = 3 → shrink until legal		6

Every element enters the window exactly once (via `right`) and leaves at most once (via `left`). No element is ever recounted from scratch — that's what turns this into a single linear pass.

Python Code

```
from collections import defaultdict

def max_subarray_length(nums: list[int], k: int) -> int:
    counts = defaultdict(int)
    left = 0
    best = 0

    for right, value in enumerate(nums):
        counts[value] += 1

        while counts[value] > k:
            counts[nums[left]] -= 1
            left += 1

        best = max(best, right - left + 1)
```

```

        best = max(best, right - left + 1)

    return best

```

Code Walkthrough

- `counts = defaultdict(int)` : tracks how many times each value currently sits inside the window. `defaultdict` avoids manual key-existence checks.
- `left = 0` : the window's left boundary; it only ever moves forward.
- `for right, value in enumerate(nums)` : `right` is both the loop index and the window's right boundary.
- `counts[value] += 1` : the new element enters the window, so its count goes up.
- `while counts[value] > k` : not an `if` — because a single shrink step might not be enough to fix the violation if the window had prior duplicates near the left edge.
- `counts[nums[left]] -= 1; left += 1` : remove the leftmost element from the count map, then advance `left`.
- `best = max(best, right - left + 1)` : once the window is legal, record its size if it's the largest seen so far.

Dry Run

Array: `[1, 2, 1, 2, 1, 2, 1, 2]`, `k = 1`

	right	value	counts after add	shrink?	left after	window size	best
0	1		{1:1}	no	0	1	1
1	2		{1:1, 2:1}	no	0	2	2
2	1		{1:2, 2:1}	yes → drop nums[0]=1	2	2	2
3	2		{1:1, 2:2}	yes → drop nums[1]=2	2	2	2
4	1		{1:2, 2:1}	yes → drop nums[2]=1	3	2	2
5	2		{1:1, 2:2}	yes → drop nums[3]=2	4	2	2
6	1		{1:2, 2:1}	yes → drop nums[4]=1	5	2	2
7	2		{1:1, 2:2}	yes → drop nums[5]=2	6	2	2

Final answer: `2`. With `k = 1`, the window can never grow past two elements once alternating duplicates start appearing.

Edge case: `[5, 5, 5, 5, 5, 5, 5]`, `k = 4`. The window grows until `counts[5]` hits `5`, at which point it shrinks back down to `4`. It can never exceed that again — answer is `4`.

Complexity Analysis

Time: $O(n)$. Each index is added to `counts` exactly once via `right`, and removed at most once via `left`. Since both pointers only move forward and never reset, the total work across the entire pass is linear, even though the inner `while` loop looks like it could be expensive.

Space: $O(n)$ in the worst case, if every element in the array is distinct and all end up as keys in `counts`.

These bounds are correct because the two pointers never move backward — there's no scenario where an element is reprocessed, which is what keeps this from degrading into the brute-force quadratic behavior.

Alternative Solutions

You could theoretically use a `Counter` from `collections` with a sorted structure to track the maximum frequency at each step, but it's unnecessary — you never need to know *which* value has the highest count, only whether the value you just added crossed the threshold. Adding a heap or sorted container here is over-engineering: it adds $O(\log n)$ factors for information the algorithm doesn't need. The plain dictionary approach is both simpler and strictly faster.

Edge Cases

- **Empty array:** returns `0` — the loop never executes, and `best` stays at its initial value.
- **All identical values with k larger than the array length:** the whole array is the answer, since the window never needs to shrink.
- **$k = 0$:** every window becomes illegal the moment any value is added twice; if k is `0` and any value must appear, behavior depends on whether the problem guarantees $k \geq 1$ — always check constraints before assuming this can't happen.
- **Single-element array:** returns `1` regardless of k , as long as $k \geq 1$.
- **All unique values:** the entire array is always legal, so the answer equals the array's length.

Common Mistakes

1. **Using `if` instead of `while` when shrinking.** A single removal might not be enough to make the window legal again.

2. **Forgetting to decrement the count when the left pointer moves.** This leaves stale counts and produces wrong shrink decisions later.
3. **Recomputing the window's frequency from scratch on every step**, which silently reintroduces the brute-force quadratic behavior.
4. **Using a `set` instead of a dictionary.** A set can tell you a value is present, but not how many times — and the shrink condition depends entirely on the exact count.
5. **Updating `best` before the window is confirmed legal**, which can record an invalid window size.

Interview Questions

- How would you modify this to return the actual subarray, not just its length?
- What changes if the constraint were "at least K" instead of "at most K"?
- Can you solve this with a fixed-size window instead of a variable one? Why or why not?
- How would this scale if the array were a live stream instead of a fixed array?
- What's the time complexity if `k` were allowed to be different for each distinct value?

Similar Problems

- **Longest Substring Without Repeating Characters (LeetCode 3):** same sliding window and count-map pattern, with `k = 1`.
- **Fruit Into Baskets (LeetCode 904):** longest subarray with at most 2 distinct values — a variant constraint on the same window.
- **Max Consecutive Ones III (LeetCode 1004):** longest window under a different kind of running constraint (flips allowed).
- **Subarrays with K Different Integers (LeetCode 992):** uses the same window mechanics to count subarrays rather than find the longest one.

Key Takeaways

Whenever a problem asks for the longest or shortest stretch that stays legal under some running rule, reach for two pointers plus a count map before anything more complex. Grow the window from the right, shrink it from the left only when necessary, and let the dictionary tell you instantly when the rule has been broken. That single pattern — no recounting, no restarting — collapses what looks like a quadratic problem into a clean linear pass.

Watch the Video

For the full walkthrough with diagrams, live dry runs, and the quiz that tests whether the pattern has actually clicked, watch the video here: {LINK}

About the Series

This breakdown is part of *Fun with Learning Technology*, a series focused on building real intuition behind coding interview patterns — not just memorizing solutions, but understanding why they work so the pattern transfers to problems you've never seen before.

Call To Action

If this helped the pattern click, consider watching the full video on YouTube and subscribing — it helps more than you'd think for a small channel. Subscribe to the Substack for written breakdowns like this one, drop a comment if something felt unclear or you want a specific problem covered next, and share it with anyone prepping for interviews.

Quick Review

You need max window length but can't just track distinct counts — which pattern?

Sliding window with a frequency hashmap; shrink from the left when any element's count exceeds K.

Time complexity of the sliding-window frequency-map approach?

$O(n)$ — each index enters and leaves the window at most once.

Space complexity for tracking frequencies?

$O(n)$ worst case (or $O(\text{distinct values})$) for the hashmap of counts.

Common bug when shrinking the window here?

Decrementing count but forgetting to delete the key at 0, or comparing max length before shrinking finishes.

This pattern vs. 'longest substring with at most K distinct chars'?

Same shrink-on-violation shape, but here the violation is a count cap ($>K$ occurrences), not distinct-count cap.

Interview follow-up: why not brute-force all subarrays?

$O(n^2)$ or worse since you'd recheck frequency validity per subarray; sliding window reuses prior work in $O(n)$.

Length of Longest Subarray With at Most K Frequency — free Python cheat sheet